Modal and vLLM Runbook

Companion Guide for InferTutor Arena

Vizuara AI Labs - Inference Engineering Workshop

Modal and vLLM Runbook

Companion Guide for InferTutor Arena

This document explains the infrastructure used in the capstone. It is intentionally detailed so that setup does not become the bottleneck.

What Is Modal?

Modal is a serverless compute platform for running Python functions on CPUs and GPUs. For this assignment, Modal gives us:

- On-demand H100 GPU containers.
- Persistent volumes for Hugging Face and vLLM caches.
- Secrets for storing Hugging Face tokens.
- A public HTTPS endpoint for the vLLM server.
- Replica scaling through min_containers and max_containers.
- Per-second billing, which is useful for short experiments.

In this capstone, your laptop runs the load tester. Modal runs the GPU-backed vLLM server.

```
Your laptop
  load_test_infertutor.py
      |
      | HTTPS streaming requests
      v
Modal web endpoint
  vLLM OpenAI-compatible server
      |
      v
H100 GPU container
  Qwen/Qwen3-VL-4B-Instruct
```

What Is vLLM?

vLLM is an inference engine for serving large language models efficiently. It provides:

- OpenAI-compatible HTTP API.
- Continuous batching.
- PagedAttention.
- Prefix caching.
- Chunked prefill.
- Tensor parallelism.
- CUDA graph / compilation support for some workloads.
- Multimodal support for models such as Qwen-VL.

The assignment uses vLLM as a production-style serving engine. You are not calling model.generate() in a notebook. You are running a server and measuring it under concurrent load.

Setup Checklist

1. Install local dependencies

Use Python 3.11 or newer.

```
pip install -r requirements.txt
```

2. Authenticate Modal

If you have never used Modal:

```
modal token new
```

If your instructor gives you a Modal token id and secret:

```
modal token set --token-id <TOKEN_ID> --token-secret <TOKEN_SECRET>
```

Check that it works:

```
modal token info
```

3. Create Hugging Face secret

The vLLM container needs a Hugging Face token to download the model.

```
modal secret create huggingface HF_TOKEN=<YOUR_HF_TOKEN>
```

The starter app expects the secret name to be:

```
huggingface
```

4. Run a smoke test

Start small:

```
python run_infertutor_experiment.py \
  --label smoke \
  --gpu-type H100 \
  --replicas 1 \
  --mode text \
  --users 5 \
  --duration 30 \
  --ramp-up 5 \
  --max-tokens 64
```

If this passes with zero errors, your infrastructure is working.

Main Commands

Single-GPU baseline

```
python run_infertutor_experiment.py \
  --label baseline-text \
  --gpu-type H100 \
  --replicas 1 \
  --max-seqs 32 \
  --max-batch-tokens 4096 \
  --mode text \
  --users 50 \
  --duration 60 \
  --ramp-up 15 \
  --max-tokens 96
```

Multimodal product baseline

```
python run_infertutor_experiment.py \
  --label mixed-r2 \
  --gpu-type H100 \
  --replicas 2 \
  --max-seqs 32 \
  --max-batch-tokens 4096 \
  --mode mixed \
  --users 100 \
  --duration 90 \
  --ramp-up 25 \
  --max-tokens 96
```

Text speed baseline with compiled mode

```
python run_infertutor_experiment.py \
  --label compiled-r4 \
  --gpu-type H100 \
  --replicas 4 \
  --no-fast-boot \
  --max-seqs 32 \
  --max-batch-tokens 4096 \
  --mode text \
  --users 400 \
  --duration 90 \
  --ramp-up 40 \
  --max-tokens 96
```

--no-fast-boot disables eager mode. Startup takes longer, but text throughput can improve substantially after vLLM compiles and captures CUDA graphs.

Important vLLM Knobs

--max-seqs

Controls how many sequences vLLM can process in a batch.

Higher values can improve throughput but may increase inter-token latency.

Good starting values:

```
32, 64, 128
```

Internal dry run result: 32 was better than 64 for the streamed tutor workload.

--max-batch-tokens

Controls the maximum total tokens in a scheduling batch.

Higher values can improve prefill throughput but may allow prefill work to interfere with decode responsiveness.

Good starting values:

```
2048, 4096, 8192
```

Internal dry run result: 4096 was better than 8192 for the main workload.

--max-model-len

Controls maximum context length. Higher values reserve more KV capacity.

For this assignment, use:

```
8192
```

You can experiment with lower values if your prompt set fits, but do not reject official benchmark prompts.

Prefix caching

Enable:

```
--prefix-cache
```

Disable:

```
--no-prefix-cache
```

Why it matters: every request shares the same InferTutor system prompt. vLLM can reuse KV blocks for repeated prefixes.

Chunked prefill

Enable:

```
--chunked-prefill
```

Disable:

```
--no-chunked-prefill
```

Why it matters: mixed workloads include short questions, image requests, and long prompts. Chunked prefill prevents long prefills from monopolizing the scheduler.

Eager vs compiled mode

Fast startup / eager mode:

```
--fast-boot
```

Compiled mode:

```
--no-fast-boot
```

Internal dry run:

- Compiled mode was excellent for text-only serving.
- Compiled mode was poor for mixed multimodal traffic in this setup.
- Eager mode was more reliable for image and mixed workloads.

Approximate Modal Cost

Modal pricing changes over time, so check Modal's current GPU pricing before the workshop. For planning, assume an H100 costs roughly a few dollars per GPU-hour.

The rough formula is:

```
Cost = GPU_count * runtime_hours * price_per_GPU_hour
```

Example estimates if H100 is around 4 USD per GPU-hour:

Run	GPU count	Duration	Approx cost
Smoke test	1	5 minutes	about 0.33 USD
Single-GPU experiment	1	10 minutes	about 0.67 USD
4-GPU run	4	10 minutes	about 2.67 USD
8-GPU boss run	8	10 minutes	about 5.33 USD

Cold starts, model downloads, and compilation time count. Stop unused deployments after experiments.

Useful command:

```
modal app list
modal app stop <APP_ID_OR_NAME>
```

Each experiment deploys a separately named app (infertutor-<label>) with min_containers equal to

--replicas. Containers with min_containers set never scale down on their own, so they bill until the app is stopped. The runner therefore stops the app automatically when a run finishes or fails. Pass --keep-app (or --deploy-only) to keep it running for reuse with --url, and stop it yourself afterwards. If a run is killed hard (e.g. the terminal is closed), check modal app list for leftovers.

Reading Results

Each load test writes a JSON file under:

```
results_infertutor/
```

Key fields:

Field	Meaning
ttft_p95_ms	p95 time to first stream
itl_p95_ms / itl_p99_ms	p95 / p99 over every ind
itl_request_mean_p95_ms	p95 of each request's av
latency_p95_ms	p95 total request latenc
aggregate_stream_chunks_	streamed content chunks
requests_per_s	successful requests per
error_rate	failed requests divided
error_kinds	error counts by type (ht
measured_window_s	length of the steady-sta
output_tokens_per_s	exact output tokens per
tpot_p95_ms	p95 time per output toke
slo_attainment / slo_goo	share and rate of reques
by_kind	all of the above split b
client	load-generator health: e
server_metrics	per-replica vLLM and GPU

Requests still running when the window ends get up to --drain-timeout seconds (default 30) to finish. Any still unfinished count as unfinished_after_drain errors, so the slowest requests are not silently dropped.

Server metrics

Each replica runs a small sampler that records vLLM's /metrics and nvidia-smi every 2 s into the infertutor-metrics Modal Volume. It also records the vLLM, CUDA and GPU driver versions and the replica's cold-start time. After the load test the runner downloads these files next to the results JSON and adds a server_metrics summary:

Field	Question it answers
requests_waiting_max vs	Is TTFT high because req
queue_time_s_mean	How long requests wait b
kv_cache_usage_max	Is KV cache memory runni
prefix_cache_hit_rate	Is prefix caching reusin
load_share_per_replica	Is traffic spread evenly
gpu_util_pct_mean	Is the GPU actually busy
cold_start_s_max, versio	How long replicas took t

To summarize metrics by hand: python server_metrics.py <results.json> <metrics_dir>.

Metrics cover only the steady-state window: requests that start after --ramp-up ends, until --duration. Ramp-up requests are counted separately in ramp_phase_requests and do not affect the score. Before measuring, the runner sends warmup bursts big enough to reach every replica (--warmup-rounds), so cold starts don't land inside the window.

score_infertutor.py marks a run ok = no if it is not comparable to official runs, for example: a modified prompt set, non-default --max-tokens or pause settings, long/image diagnostic modes, an endpoint the runner did not deploy (GPU count self-reported), or a window shorter than 30 s. The local score does not include quality_pass_rate.

For final judging, your instructor may run the same endpoint with a hidden evaluator that computes exact token throughput and quality pass rate.

Suggested Workflow

1. Run the smoke test. 2. Run the 1-GPU baseline. 3. Increase users until p95 TTFT bends. 4. Toggle prefix caching. 5. Toggle chunked prefill. 6. Try max-seqs and max-batch-tokens sweeps. 7. Add replicas. 8. Try compiled mode for text-only. 9. Pick your final run. 10. Stop Modal apps. 11. Write your engineering report.

What Good Engineering Looks Like

Good submissions do not just show a high number. They explain the path:

- What bottleneck did you think you had?
- What metric supported that hypothesis?
- What knob did you change?
- What happened?
- Why did you keep or reject that configuration?

The goal is not magic. The goal is disciplined measurement.